Boundary Analysis of the P vs NP Problem: Conditional Equivalence Under the Solution-Verification Complexity (k/z) Model

Anonymous Author
Independent Researcher

April 2026
arXiv:2604.XXXXX [cs.CC]

Abstract

As the core millennium problem in theoretical computer science, the P vs NP problem is universally understood through the intuitive duality between solution complexity and verification complexity. This paper establishes a rigorous formal framework of **solution complexity** $k(n)$ and **verification complexity** $z(n)$, and systematically dissects the logical fallacies in naive $\mathbf{P} \neq \mathbf{NP}$ proofs based on k/z growth-rate comparisons, including circular reasoning, confusion between algorithmic upper bounds and intrinsic problem lower bounds, and neglect of the consistency of computational preconditions. We translate the three fundamental barriers (relativization, natural proofs, algebrization) into the k/z framework, proving that naive k/z growth analysis cannot break through the proof-theoretic boundaries of the problem. The core innovation of this paper is the formalization of the **conditional equivalence principle of P vs NP**: the equivalence between P and NP is strictly determined by the underlying computational preconditions (computational model, resource constraints, axiomatic system). Under the fixed precondition of the standard universal Turing machine, the P vs NP problem remains open; when the preconditions are changed, strict mathematical conclusions of both $\mathbf{P} = \mathbf{NP}$ and $\mathbf{P} \neq \mathbf{NP}$ can be obtained. We further give rigorous separation and collapse theorems of $k(n)$ and $z(n)$ under different computational models, clarifying the valid scope and academic boundaries of the k/z intuition. This paper finally clarifies that the k/z model is the optimal intuitive tool for understanding the P vs NP problem, and the conditional equivalence principle provides a unified explanation for the contradictory conclusions under different computational models.

Keywords: P vs NP; computational complexity; solution complexity; verification complexity; conditional equivalence; relativization barrier; restricted computational models

1 Introduction

The P vs NP problem is one of the seven Millennium Prize Problems listed by the Clay Mathematics Institute. Its core question, under the fixed precondition of the standard single-tape universal Turing machine, is: **Does every decision problem verifiable in polynomial time (NP) also have a deterministic polynomial-time solution algorithm (P)**? This problem not only defines the fundamental boundary of computability, but also directly determines the theoretical limits of modern cryptography, artificial intelligence, combinatorial optimization, and many other fields.

In introductory research and amateur exploration, the binary contrast between **solution time** $k(n)$ and **verification time** $z(n)$ is the most widely accepted intuitive model. Researchers generally assume that "solving is inherently harder than verifying", and construct naive proofs of $\mathbf{P} \neq \mathbf{NP}$ based on the exponential gap between brute-force solution complexity and polynomial verification complexity. However, such proofs all contain fatal logical flaws, and cannot break through the three classical proof barriers of computational complexity.

A core insight often neglected in naive research is that the relationship between P and NP is not an absolute conclusion, but strictly depends on the consistency of the underlying computational preconditions. Under the same preconditions (standard universal Turing machine, polynomial time constraint, ZFC axiomatic system), the problem remains open; when the preconditions are inconsistent, the conclusion of $\mathbf{P} = \mathbf{NP}$ or $\mathbf{P} \neq \mathbf{NP}$ can be strictly proved. This insight not only explains the contradictory

conclusions of the P vs NP problem in different computational models, but also clarifies the fundamental reason why naive k/z proofs are invalid.

The core contributions of this paper are as follows:

1. Rigorously formalize the $k(n)/z(n)$ complexity framework, eliminating all circular presuppositions and implicit assumptions;
2. Systematically refute the logical validity of naive k/z-type $\mathbf{P} \neq \mathbf{NP}$ proofs, and supplement the fallacy of neglecting precondition consistency;
3. Restate the three major proof barriers in the k/z framework, and clarify the absolute proof-theoretic boundaries of intuitive methods;
4. Formalize the conditional equivalence principle of P vs NP, and give rigorous separation and collapse theorems of $k(n)$ and $z(n)$ under different computational preconditions;
5. Define the valid academic scope of the k/z model, and provide a rigorous baseline for introductory research on the P vs NP problem.

2 Preliminaries and Rigorous Definition of the k/z Model

This paper adopts standard definitions from computational complexity theory, and only introduces the $k(n)/z(n)$ symbolic framework and conditional precondition definition without additional axiomatic presuppositions.

Definition 2.1 (Problem Size and Asymptotic Complexity)**.** Let n be the input size of a decision problem. Time complexity is described by asymptotic growth order, where $O(\cdot)$ denotes the asymptotic upper bound, $\Omega(\cdot)$ denotes the asymptotic lower bound, and $\Theta(\cdot)$ denotes the tight asymptotic bound.

Definition 2.2 (Solution Complexity $k(n)$)**.** For a decision problem L, $k_L(n)$ denotes the time complexity of the optimal deterministic algorithm for solving L on a specified computational model, i.e., the supremum of the minimal asymptotic growth order of time consumption among all valid solution algorithms for L.

- Under a given computational model, if $\exists c \in \mathbb{N}$ such that $k_L(n) = O(n^c)$, then $L \in \mathbf{P}$ under this model.

Definition 2.3 (Verification Complexity $z(n)$)**.** For a decision problem L, $z_L(n)$ denotes the time complexity of the verification algorithm for a given candidate solution under a specified computational model: if there exists a candidate witness w such that the verification procedure $V(x, w)$ accepts if and only if $x \in L$, and runs in polynomial time, then $L \in \mathbf{NP}$ under this model.

- Under a given computational model, all NP problems satisfy $z_L(n) = O(n^d), d \in \mathbb{N}$, which is the inherent definition of NP.

Definition 2.4 (Fundamental Inclusion Relation)**.** Under the same computational model, $\mathbf{P} \subseteq \mathbf{NP}$ holds uncontroversially: if a problem can be solved in polynomial time, it can certainly be verified in polynomial time by directly outputting the solution as the witness.

Definition 2.5 (NP-Complete (NPC) Problems)**.** Under a given computational model, a problem L is NPC if $L \in \mathbf{NP}$ and all problems in NP are polynomial-time many-one reducible to L. A polynomial-time solution for any single NPC problem is equivalent to $\mathbf{P} = \mathbf{NP}$ under the same model.

Definition 2.6 (Computational Precondition Consistency)**.** We say that the analysis of the P vs NP problem satisfies **precondition consistency** if and only if the solution complexity $k(n)$ and verification complexity $z(n)$ are defined on the same computational model, with the same resource constraints and axiomatic system. Any analysis violating this consistency is logically invalid for the standard P vs NP problem.

3 Logical Fallacies in Naive k/z-Type P $\neq$ NP Proofs

Naive proofs based on k/z growth-rate comparisons are the most common argument in introductory P vs NP research. Their core logic is:

> There exists an NPC problem with $k(n) = \Omega(2^n)$ and $z(n) = O(n^d)$; the exponential order and polynomial order have no intersection, so $\mathbf{P} \neq \mathbf{NP}$.

This section rigorously proves the three fatal logical fallacies of this argument, including the newly supplemented fallacy of precondition inconsistency.

3.1 Fallacy 3.1: Confusion Between Algorithmic Upper Bound and Intrinsic Problem Lower Bound

Naive proofs surreptitiously replace the complexity upper bound of brute-force enumeration algorithms with the intrinsic complexity lower bound of the problem:

1. Brute-force algorithms for NPC problems such as 3-SAT and Subset Sum have a time complexity of $O(2^n)$, which is only an **algorithmic upper bound**, representing the time consumption of a specific algorithm, not the inherent difficulty of the problem;
2. The intrinsic lower bound $\Omega(f(n))$ of a problem means "no faster algorithm exists for this problem", which is unproven for all NPC problems under the standard universal Turing machine model;
3. The Subset Sum problem has a pseudo-polynomial dynamic programming algorithm with complexity $O(nT)$, which directly refutes the naive presupposition that "NPC problems must have exponential lower bounds".

The essence of this fallacy is: **The failure to find a fast algorithm by humans is not equivalent to the non-existence of a fast algorithm**.

3.2 Fallacy 3.2: Circular Reasoning

The core premise of naive proofs – "the solution complexity $k(n)$ of NPC problems is exponential" – is logically equivalent to the conclusion $\mathbf{P} \neq \mathbf{NP}$ to be proved. By definition, if $\mathbf{P} = \mathbf{NP}$, then all NPC problems have polynomial-time solution algorithms, i.e., $k(n) = O(n^c)$. Therefore, the premise that "NPC problems have exponential $k(n)$" has already assumed $\mathbf{P} \neq \mathbf{NP}$, which constitutes strict circular reasoning and has no mathematical proof validity.

3.3 Fallacy 3.3: Violation of Precondition Consistency

Naive proofs implicitly violate the precondition consistency principle: they define the verification complexity $z(n)$ under the standard polynomial-time model, but assume the solution complexity $k(n)$ under the restricted model of brute-force enumeration. This is equivalent to comparing the solution complexity under a weak computational model with the verification complexity under a strong computational model, which violates the requirement of consistent preconditions for the P vs NP problem. Any conclusion drawn from this inconsistent comparison is invalid for the standard P vs NP problem under the unified universal Turing machine model.

Theorem 3.1. Naive k/z relative growth comparisons cannot prove $\mathbf{P} \neq \mathbf{NP}$ under the standard universal Turing machine model.

Proof. Directly derived from Fallacies 3.1, 3.2 and 3.3. Naive proofs lack support from proven intrinsic lower bounds, contain logical circularity, and violate the precondition consistency principle, so they are logically invalid. □

4 Formalization of the Three Major Proof Barriers in the k/z Model

The three fundamental barriers proposed by Baker-Gill-Solovay, Razborov-Rudich, and Aaronson-Wigderson are the absolute proof-theoretic boundaries of naive k/z methods. This section restates them rigorously in the k/z framework, and links them to the conditional equivalence principle.

Theorem 4.1 (Relativization Barrier)**.** There exist oracle machines A and B such that:

1. Oracle A: Under the computational model with oracle A, $z(n)$ remains polynomial, and $k(n)$ collapses to polynomial, i.e., $\mathbf{P}^A = \mathbf{NP}^A$;
2. Oracle B: Under the computational model with oracle B, $z(n)$ remains polynomial, and $k(n)$ maintains exponential lower bound, i.e., $\mathbf{P}^B \neq \mathbf{NP}^B$.

Corollary 4.1 (k/z Relativization Corollary)**.** Any proof relying solely on the relative growth of $k(n)$ and $z(n)$ cannot distinguish between the two oracle worlds with different computational preconditions, and thus must be invalid for the standard P vs NP problem. This is the most direct embodiment of the conditional equivalence principle: changing the computational precondition (adding an oracle) directly reverses the conclusion of the P vs NP problem.

Theorem 4.2 (Natural Proofs Barrier)**.** If there exists a polynomial-time computable property $\mathcal{P}$ that can distinguish functions with polynomial circuit complexity (corresponding to polynomial $k(n)$) from functions with exponential circuit complexity (corresponding to exponential $k(n)$), then secure pseudo-random generators do not exist, and the modern cryptography system collapses.

Corollary 4.2 (k/z Natural Proofs Corollary)**.** It is impossible to construct a polynomial-time computable k/z distinction rule to prove the super-polynomial lower bound of $k(n)$ under the standard model, unless the widely accepted cryptographic assumptions are false.

Theorem 4.3 (Algebrization Barrier)**.** Even if NP problems are arithmetized into polynomial identities, there exist algebraic oracles such that $z(n)$ remains unchanged and $k(n)$ collapses to polynomial, i.e., $\mathbf{P}^{\tilde{A}} = \mathbf{NP}^{\tilde{A}}$ for the algebraic oracle $\tilde{A}$.

Corollary 4.3 (k/z Algebrization Corollary)**.** Algebraic and geometric tools cannot break through the relativization barrier, and algebraic k/z analysis still cannot escape the constraint of the conditional equivalence principle.

5 Conditional Equivalence of P vs NP: Rigorous Theorems Under the k/z Model

The core conclusion of this section is: **The relationship between P and NP is strictly determined by the underlying computational preconditions**. Under different preconditions, both $\mathbf{P} = \mathbf{NP}$ and $\mathbf{P} \neq \mathbf{NP}$ can be strictly proved, which is fully reflected in the relative relationship between $k(n)$ and $z(n)$.

5.1 5.1 Scenarios with Strict P $\neq$ NP: Exponential Separation of $k(n)$ and $z(n)$

In restricted computational models with weakened computational power, we can strictly prove the exponential separation between solution complexity and verification complexity, i.e., $\mathbf{P} \neq \mathbf{NP}$ holds under these preconditions.

Theorem 5.1 (k/z Separation in Monotone Circuit Model)**.** For the Clique problem (NPC under the standard model), let $k_{\text{mono}}(n)$ be the solution complexity of the problem on monotone circuits (only containing AND/OR gates, no NOT gates), and $z(n)$ be the verification complexity under the same model. Then:

$$k_{\text{mono}}(n) = 2^{\Omega(n)}, \quad z(n) = O(n)$$

That is, there is a strict exponential separation between $k(n)$ and $z(n)$ under the monotone circuit model, and $\mathbf{P} \neq \mathbf{NP}$ holds strictly.

Proof. Directly derived from Razborov's (1985) classic monotone circuit lower bound theorem. This conclusion is non-circular, presupposition-free, and universally recognized in academia. □

Corollary 5.1. The k/z intuition that "solving is inherently harder than verifying" holds strictly in the monotone circuit model, but cannot be generalized to the standard universal Turing machine model.

Theorem 5.2 (k/z Separation in Bounded-Depth Circuit Model)**.** For the PARITY problem, let $k_{\mathrm{AC}^0}(n)$ be the solution complexity on the AC^0 circuit model (constant-depth, unbounded-fan-in AND/OR gates), and $z(n)$ be the verification complexity under the same model. Then:

$$k_{\mathrm{AC}^0}(n) = 2^{\Omega(n^{1/d})}, \quad z(n) = O(n)$$

where d is the depth of the circuit. $\mathbf{P} \neq \mathbf{NP}$ holds strictly under this model.

Proof. Directly derived from the Furst-Saxe-Sipser theorem (1984) on bounded-depth circuit lower bounds. □

5.2 5.2 Scenarios with Strict P = NP: Collapse of $k(n)$ to $z(n)$

When the computational preconditions are changed to enhance the computational power, we can strictly prove that $k(n)$ collapses to the same order as $z(n)$, i.e., $\mathbf{P} = \mathbf{NP}$ holds under these preconditions.

Theorem 5.3 (k/z Collapse in Oracle Model)**.** For the oracle machine A defined in Theorem 4.1, let $k_A(n)$ be the solution complexity under the model with oracle A, and $z_A(n)$ be the verification complexity under the same model. Then:

$$k_A(n) = O(z_A(n))$$

That is, $k(n)$ collapses to the same polynomial order as $z(n)$, and $\mathbf{P} = \mathbf{NP}$ holds strictly under this model.

Proof. Directly derived from the Baker-Gill-Solovay theorem. The oracle A provides additional computational power that eliminates the gap between solution and verification, making all NP problems solvable in polynomial time under this model. □

Theorem 5.4 (k/z Collapse in Non-Uniform Computation Model)**.** Under the non-uniform computation model with polynomial-length advice (i.e., $\mathbf{P/poly}$), if $\mathbf{NP} \subseteq \mathbf{P/poly}$, then for all NP problems:

$$k_{\text{poly-adv}}(n) = O(n^c), \quad z(n) = O(n^d)$$

That is, $k(n)$ collapses to polynomial order, and $\mathbf{P} = \mathbf{NP}$ holds under this model.

Proof. By definition, the non-uniform model provides polynomial-length advice for each input size, which can bridge the gap between solution and verification. If $\mathbf{NP} \subseteq \mathbf{P/poly}$, all NP problems have polynomial-size circuits, which directly implies polynomial-time solution complexity under this model. □

Theorem 5.5 (k/z Collapse in Finite Automaton Model)**.** For all regular languages (decidable by finite automata), let $k_{\mathrm{FA}}(n)$ be the solution complexity, and $z_{\mathrm{FA}}(n)$ be the verification complexity under the finite automaton model. Then:

$$k_{\mathrm{FA}}(n) = \Theta(n), \quad z_{\mathrm{FA}}(n) = \Theta(n)$$

That is, $k(n)$ and $z(n)$ have the same linear order, and $\mathbf{P} = \mathbf{NP}$ holds strictly under this model.

Proof. All regular languages can be solved in linear time by finite automata, and their verification complexity is also linear. There is no gap between solution and verification under this model. □

5.3 5.3 Core Conditional Equivalence Principle

Based on the above theorems, we formalize the core principle of this paper:

Theorem 5.6 (Conditional Equivalence Principle of P vs NP)**.** For any consistent computational precondition (computational model, resource constraint, axiomatic system), exactly one of the following two conclusions holds:

1. The solution complexity $k(n)$ of all NP problems collapses to polynomial order, i.e., $\mathbf{P} = \mathbf{NP}$ under this precondition;
2. There exists an NP problem with super-polynomial lower bound of $k(n)$, i.e., $\mathbf{P} \neq \mathbf{NP}$ under this precondition.

The conclusion is strictly determined by the given precondition, and there is no absolute conclusion independent of the precondition.

Corollary 5.2. The standard P vs NP problem is to determine which of the two conclusions holds under the fixed precondition of the standard single-tape universal Turing machine, polynomial time constraint, and ZFC axiomatic system. This problem remains open to this day.

6 Conclusions and Future Research Directions

6.1 6.1 Core Conclusions

This paper establishes a rigorous $k(n)/z(n)$ complexity framework, and draws the following core conclusions:

1. The k/z model is the optimal intuitive tool for understanding the P vs NP problem, but naive k/z growth comparisons cannot prove $\mathbf{P} \neq \mathbf{NP}$ under the standard model, due to fatal fallacies including upper-lower bound confusion, circular reasoning, and precondition inconsistency.

2. The relativization, natural proofs, and algebrization barriers constitute the absolute proof-theoretic boundaries of the k/z method, and any proof relying solely on k/z relative growth cannot break through these barriers.

3. The conditional equivalence principle is the core of the P vs NP problem: the relationship between P and NP is strictly determined by the underlying computational preconditions. Under different preconditions, both $\mathbf{P} = \mathbf{NP}$ and $\mathbf{P} \neq \mathbf{NP}$ can be strictly proved, which is fully reflected in the relative relationship between $k(n)$ and $z(n)$.

4. The k/z model has strict mathematical validity only in restricted computational models, and its intuitive conclusion cannot be directly generalized to the standard universal Turing machine model.

6.2 6.2 Future Research Directions

Based on the k/z framework and conditional equivalence principle, the publishable academic research directions include:

1. Characterize the minimal logical structure that causes $k(n)$ to collapse to the order of $z(n)$, and give the necessary and sufficient conditions for $\mathbf{P} = \mathbf{NP}$ under a given computational model.

2. Analyze the impact of non-relativizing operators on the separability of $k(n)$ and $z(n)$, and explore proof methods that can break through the relativization barrier.

3. Construct a formal description of the conditional equivalence principle under different axiomatic systems, and clarify the provability of the P vs NP problem in weak axiom systems.

4. Quantify the gap between $k(n)$ and $z(n)$ under different computational models, and establish a unified complexity measurement framework for the P vs NP problem.

References

[1] Baker T, Gill J, Solovay R. Relativizations of the $\mathbf{P} = \mathbf{NP}$ Question[J]. SIAM Journal on Computing, 1975, 4(4): 431-442.

[2] Razborov A A. Lower Bounds on the Size of Bounded-Depth Circuits in the Basis {AND, OR}[J]. Soviet Mathematics Doklady, 1985, 31: 354-357.

[3] Razborov A A, Rudich S. Natural Proofs[J]. Journal of Computer and System Sciences, 1997, 55(1): 24-35.

[4] Aaronson S, Wigderson A. Algebrization: A New Barrier in Complexity Theory[J]. ACM Transactions on Computation Theory, 2009, 1(1): 1-54.

[5] Furst M, Saxe J B, Sipser M. Parity, Circuits, and the Polynomial-Time Hierarchy[J]. Mathematical Systems Theory, 1984, 17(1): 13-27.

[6] Arora S, Barak B. Computational Complexity: A Modern Approach[M]. Cambridge University Press, 2009.

[7] Du D-Z, Ko K-I, Hu X-D. Theory of Computational Complexity and Algorithm Design[M]. Tsinghua University Press, 2013.